

# How to Return a Value from Task in C# with Examples

In this article, I am going to discuss **How to Return a Value from a Task in C#** with Examples. Please read our previous article where we discussed **Task in C#** with Examples. At the end of this article, you will understand How to Return a Value from a Task in C# with examples.

## How to Return a Value from a Task in C#?

The .NET Framework also provides a generic version of the Task class i.e. Task<T>. Using this Task<T> class we can return data or values from a task. In Task<T>, T represents the data type that you want to return as a result of the task. With Task<T>, we have the representation of an asynchronous method that is going to return something in the future. That something could be a string, a number, a class, etc.

## Example to Understand Task<T> in C#:

Let us understand this with an example. What are we going to do is, we are going to communicate with a Web API that we are going to build and we will try to retrieve the message that we receive from the Web API.

## Creating ASP.NET Web API Project

Open Visual Studio and creates a new ASP.NET Web API Project. If you are new to ASP.NET Web API, then please have a look at our ASP.NET Web API Tutorials. Here, we are creating an Empty Web API Project with the name WebAPIDemo. Once we created the Web API Project then add a Web API Controller with the name GreetingsController inside the Controllers folder. Once you add the GreetingsController then copy and paste the following code inside it.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Net;
using System.Net.Http;
using System.Web.Http;
namespace WebAPIDemo.Controllers
{
    public class GreetingsController : ApiController
    {
        //api/greetings/name
        [Route("api/greetings/{name}")]
        [HttpGet]
        public string GetGreetings(string name)
        {
            return $"Hello {name}, Welcome to Web API";
        }
    }
}
```

Now, run the Web API application, and you can access the GetGreetings resource using the URL [api/greetings/name](#) as shown in the below image. Please note the port number, it might be different in your case.



Once you run the Web API Project, then you can access the above resource from anywhere. You can access it from a web browser, you can access using postman and fiddler, and you can also access it from other web, windows, and console application. In our example, we are going to access this from our Console application.

The idea is that since the Web API is external to our Console application. So, talking to the Web API is an IO operation, which means that we will have to use or we should use Asynchronous Programming.

## Calling Web API HTTP Request from Console Application

Now, we will make an HTTP Request to the Web API (External Resource) from our Console Application. Please copy the endpoint address of the Web API. And then modify the code as follows. You need to replace the port number on which your Web API application is running. In the below example, we are making an asynchronous call to the Web API.

```
using System;
using System.Net.Http;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        public static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started.....");
            Console.WriteLine("Enter the Name: ");
            string Name = Console.ReadLine();
            SomeMethod(Name);
            Console.WriteLine("Main Method End");
            Console.ReadKey();
        }
        public async static void SomeMethod(string Name)
        {
            Console.WriteLine("Some Method Started.....");
            var GreetingSMessage = await Greetings(Name);
            Console.WriteLine($"\\n{GreetingSMessage}");
            Console.WriteLine("Some Method End");
        }

        public static async Task<string> Greetings(string Name)
        {
            string message = string.Empty;
```

```

        using (var client = new HttpClient())
        {
            client.BaseAddress = new Uri("http://localhost:58937/");

            HttpResponseMessage response = await
client.GetAsync($"api/greetings/{Name}");
            message = await response.Content.ReadAsStringAsync();
        }
        return message;
    }
}
}

```

**Output:** Before running the console application, please make sure your web application is running. Once your Web API Application is running, then run the console application. It will ask you to enter your name. Once you enter the name, press the enter key and you will see the following output.

```

Main Method Started.....
Enter the Name:
Pranaya
Some Method Started.....
Main Method End

"Hello Pranaya, Welcome to Web API"
Some Method End

```

The point that you need to remember is that if you are writing any asynchronous method, then you can use Task as the return type if it is not returning anything or you can use Task<T> when your method returns something. Here T can be anything like string, integer, Class, etc.

And we also saw that by using await, we are suspending the execution of the current thread. So we are freeing the thread so that the thread can be used in other parts of the application. And once we have a response, for example, from our Web API, then it will use the thread again to execute the rest part of the method.

## C# Task with Errors:

So far, every task that we have executed has been completed successfully. And, in real life, this may not be the case always. Sometimes errors will happen. For example, maybe we were wrong in the URL. In this case, we will get a 404 error. Let us understand this with an error. In the URL, I have changed greetings to greetings2 as shown in the below code. Further, I have included the **response.EnsureSuccessStatusCode();** statement to throw 404 error.

```

using System;
using System.Net.Http;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {

```

```

public static void Main(string[] args)
{
    Console.WriteLine("Main Method Started.....");
    Console.WriteLine("Enter the Name: ");
    string Name = Console.ReadLine();

    SomeMethod(Name);
    Console.WriteLine("Main Method End");
    Console.ReadKey();
}
public async static void SomeMethod(string Name)
{
    Console.WriteLine("Some Method Started.....");
    var GreetingSMessage = await Greetings(Name);
    Console.WriteLine($"\\n{GreetingSMessage}");

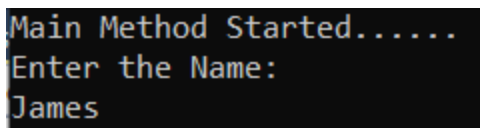
    Console.WriteLine("Some Method End");
}

public static async Task<string> Greetings(string Name)
{
    string message = string.Empty;
    using (var client = new HttpClient())
    {
        client.BaseAddress = new Uri("http://localhost:58937/");

        HttpResponseMessage response = await
client.GetAsync($"api/greetings2/{Name}");
        response.EnsureSuccessStatusCode();
        message = await response.Content.ReadAsStringAsync();
    }
    return message;
}
}
}

```

**Output:** With the above changes in place now we run the application, before that make sure the Web API Application is running. Enter the name and press the enter button as shown in the below image.

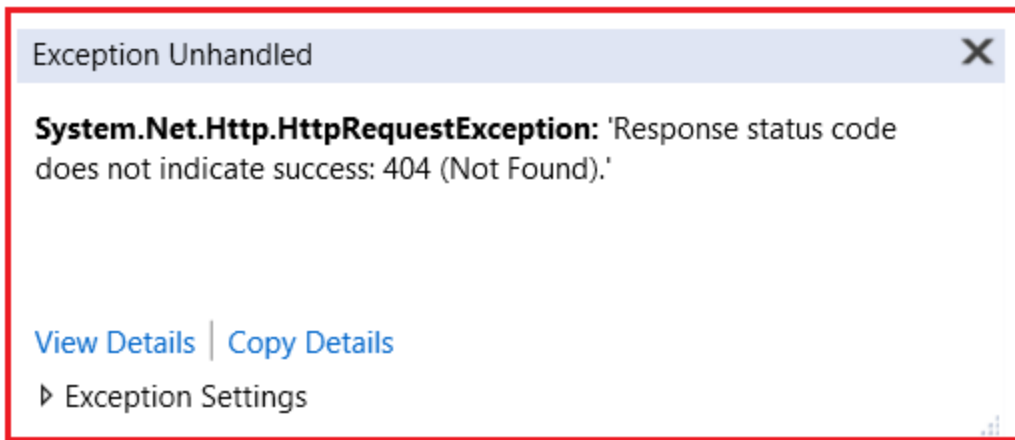


```

Main Method Started.....
Enter the Name:
James

```

Once you enter your name and press the enter button, then you will get the following unhandled exception.



Please observe here we are getting 404 Not Found HttpRequestException. This is a bad user experience. The user should not see this message. If any exception occurred then instead of showing the exception details, we should show some generic error message. Let us see how we can do this. Within the SomeMethod we need to use the Try and Catch block to handle the unhandled exception which is shown in the below example.

```
using System;
using System.Net.Http;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        public static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started.....");
            Console.WriteLine("Enter the Name: ");
            string Name = Console.ReadLine();

            SomeMethod(Name);
            Console.WriteLine("Main Method End");
            Console.ReadKey();
        }
        public async static void SomeMethod(string Name)
        {
            Console.WriteLine("Some Method Started.....");
            try
            {
                var GreetingSMessage = await Greetings(Name);
                Console.WriteLine($"\\n{GreetingSMessage}");
            }
            catch (HttpRequestException ex)
            {
                Console.WriteLine($"\\nError Occurred. {ex.Message}");
            }
            Console.WriteLine("Some Method End");
        }
    }
}
```

```

public static async Task<string> Greetings(string Name)
{
    string message = string.Empty;
    using (var client = new HttpClient())
    {
        client.BaseAddress = new Uri("http://localhost:58937/");

        HttpResponseMessage response = await
client.GetAsync($"api/greetings2/{Name}");
        response.EnsureSuccessStatusCode();
        message = await response.Content.ReadAsStringAsync();
    }
    return message;
}
}
}

```

Output:

```

Main Method Started.....
Enter the Name:
James
Some Method Started.....
Main Method End

Error Occurred. Response status code does not indicate success: 404 (Not Found).
Some Method End

```

Now, we are not getting that exception rather we seeing some generic message on the console. This is different than having an unhandled exception. So, here we completely control what was going to happen if we get an exception.

## What happens if we omit the await keyword while calling the Greetings method?

Something that you need to keep in mind is that if you don't await the task, then the exception will not be thrown to the caller method i.e. the method from where we called the async method. In our example, it will not throw the exception to the SomeMethod. Let's see that. Let's remove the await keyword and the printing of the greeting statement inside the SomeMethod as shown in the below example and run the application.

```

using System;
using System.Net.Http;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        public static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started.....");

```

```

        Console.WriteLine("Enter the Name: ");
        string Name = Console.ReadLine();

        SomeMethod(Name);
        Console.WriteLine("Main Method End");
        Console.ReadKey();
    }
    public async static void SomeMethod(string Name)
    {
        Console.WriteLine("Some Method Started.....");
        try
        {
            var GreetingSMessage = Greetings(Name);
            //Console.WriteLine($"\\n{GreetingSMessage}");
        }
        catch (HttpRequestException ex)
        {
            Console.WriteLine($"\\nError Occurred. {ex.Message}");
        }
        Console.WriteLine("Some Method End");
    }

    public static async Task<string> Greetings(string Name)
    {
        string message = string.Empty;
        using (var client = new HttpClient())
        {
            client.BaseAddress = new Uri("http://localhost:58937/");

            HttpResponseMessage response = await
client.GetAsync($"api/greetings2/{Name}");
            response.EnsureSuccessStatusCode();
            message = await response.Content.ReadAsStringAsync();
        }
        return message;
    }
}
}

```

Now, when you run the application, you will not get the exception. You will get the following output which does execute the catch block.

```

Main Method Started.....
Enter the Name:
James
Some Method Started.....
Some Method End
Main Method End

```

## Why we did not get the exception?

Please have a look at the below image. When an Exception has occurred inside an async method, that exception is encapsulated inside the Task.

The Exception is wrapped inside the Task



```
public static async Task<string> Greetings(string Name)
{
    string message = string.Empty;
    using (var client = new HttpClient())
    {
        client.BaseAddress = new Uri("http://localhost:58937/");

        HttpResponseMessage response = await client.GetAsync($"api/greetings2/{Name}");
        response.EnsureSuccessStatusCode();
        message = await response.Content.ReadAsStringAsync();
    }
    return message;
}
```

When an Exception is occurred inside a async method that exception is encapsulated inside the Task

If you want to unwrap the exception then you need to use await as shown in the below image. If you are not using await, then you will never get the exception.

Unwrapping the Exception from Task



```
var GreetingSMessage = await Greetings(Name);
```

**Note:** We can catch exceptions by using a simple try-catch block. But if we never await the task, then even if we have an exception, the exception is not going to be thrown. So, if you want to be notified about the exceptions that you may have, you need to await the task.

## Example to Understand How to Return Complex Type Value From a Task in C#:

In the below example, we are returning a Complex type.

```
using System;
using System.Threading.Tasks;
namespace TaskBasedAsynchronousProgramming
{
    class Program
```

```

{
    static void Main(string[] args)
    {
        Console.WriteLine($"Main Thread Started");
        SomeMethod();
        Console.WriteLine($"Main Thread Completed");
        Console.ReadKey();
    }
    private async static void SomeMethod()
    {
        Employee emp = await GetEmployeeDetails();
        Console.WriteLine($"ID: {emp.ID}, Name : {emp.Name}, Salary :
{emp.Salary}");
    }
    static async Task<Employee> GetEmployeeDetails()
    {
        Employee employee = new Employee()
        {
            ID = 101,
            Name = "James",
            Salary = 10000
        };
        return employee;
    }
}
public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public double Salary { get; set; }
}
}

```

**Output:**

```

Main Thread Started
ID: 101, Name : James, Salary : 10000
Main Thread Completed

```

In the next article, I am going to discuss [How to Execute Multiple Tasks in C#](#) with Examples. Here, in this article, I try to explain **How to Return a Value from a Task in C#** with Examples. I hope you enjoy this Task that Returns a Value in C# with Examples article.